In [1]:

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda x: x.view(-1))
])

train_dataset = datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=100, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=100, shuffle=False)

class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]

class CIFARMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(3072, 256)
        self.relu1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.fc2 = nn.Linear(256, 128)
        self.relu2 = nn.ReLU()
        self.drop2 = nn.Dropout(0.3)

        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.drop1(x)

        x = self.fc2(x)
        x = self.relu2(x)
        x = self.drop2(x)
```

```python
        x = self.fc3(x)
        return x

def train_one_epoch(model, loader, criterion, optimizer):
    model.train()

    total_loss = 0.0
    correct = 0
    total = 0

    for x, y in loader:
        optimizer.zero_grad()
        outputs = model(x)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)

        _, predicted = torch.max(outputs, 1)
        correct += (predicted == y).sum().item()
        total += y.size(0)

    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy

def evaluate(model, loader, criterion):
    model.eval()

    total_loss = 0.0
    correct = 0
    total = 0

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for x, y in loader:
            outputs = model(x)
            loss = criterion(outputs, y)

            total_loss += loss.item() * x.size(0)

            _, predicted = torch.max(outputs, 1)
            correct += (predicted == y).sum().item()
            total += y.size(0)

            all_preds.extend(predicted.cpu().numpy())
            all_labels.extend(y.cpu().numpy())

    avg_loss = total_loss / total
    accuracy = correct / total
    return avg_loss, accuracy, np.array(all_labels), np.array(all_preds)

model = CIFARMLP()
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.SGD(
    model.parameters(),
```

```
    lr=0.1,
    weight_decay=0.0001
)

num_epochs = 40

train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []

for epoch in range(num_epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opti
    test_loss, test_acc, _, _ = evaluate(model, test_loader, criterion)

    train_losses.append(train_loss)
    train_accuracies.append(train_acc)
    test_losses.append(test_loss)
    test_accuracies.append(test_acc)

    print(f"Epoch {epoch+1}/{num_epochs}")
    print(f"Train Loss: {train_loss:.4f}, Train Accuracy: {train_acc:.4f}")
    print(f"Test  Loss: {test_loss:.4f}, Test  Accuracy: {test_acc:.4f}")
    print("-" * 40)

epochs = range(1, num_epochs + 1)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(epochs, train_losses, label="Train Loss")
ax.plot(epochs, test_losses, label="Test Loss")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("CIFAR-10 MLP Loss")
ax.legend()
ax.grid(True)
plt.show()
plt.close(fig)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(epochs, train_accuracies, label="Train Accuracy")
ax.plot(epochs, test_accuracies, label="Test Accuracy")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.set_title("CIFAR-10 MLP Accuracy")
ax.legend()
ax.grid(True)
plt.show()
plt.close(fig)

test_loss, test_acc, y_true, y_pred = evaluate(model, test_loader, criterion)
print("Final Test Accuracy:", test_acc)

num_classes = 10
cm = np.zeros((num_classes, num_classes), dtype=np.int32)

for true_label, pred_label in zip(y_true, y_pred):
    cm[true_label, pred_label] += 1

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
```

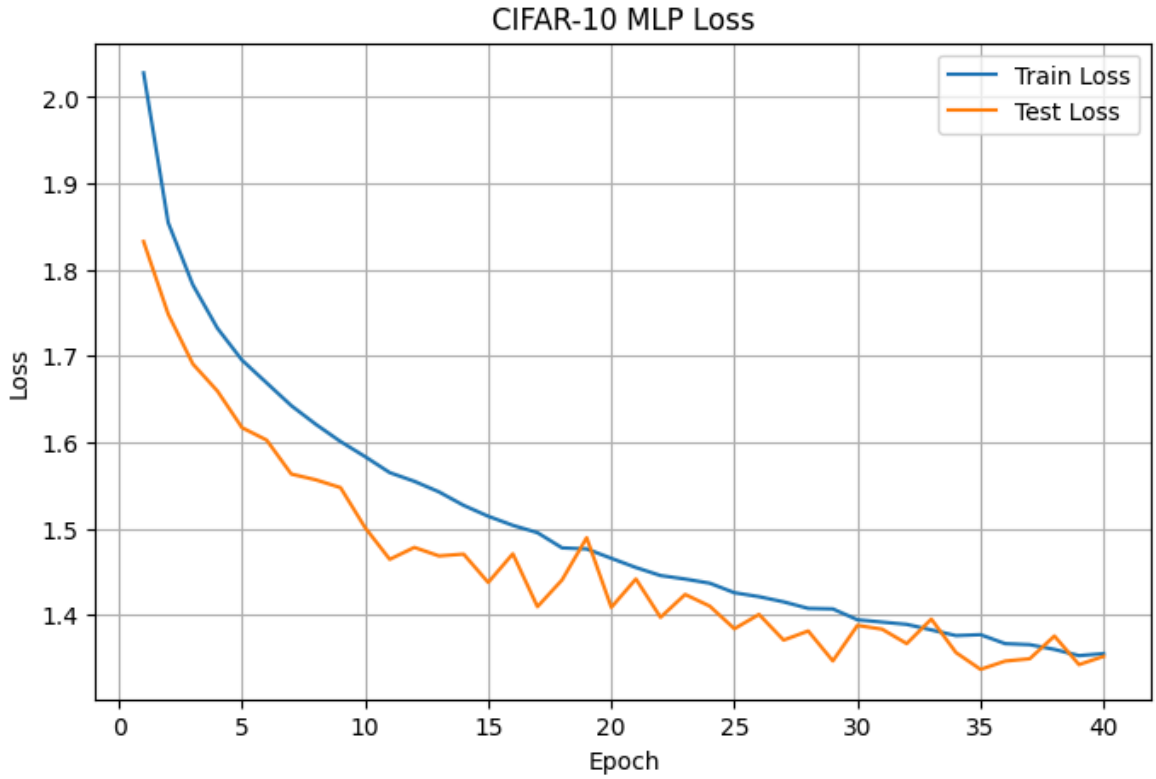
```
            xticklabels=class_names, yticklabels=class_names, ax=ax)
ax.set_xlabel("Predicted Class")
ax.set_ylabel("True Class")
ax.set_title("CIFAR-10 Confusion Matrix")
plt.show()
plt.close(fig)

most_confused_for_each_class = {}

for i in range(num_classes):
    row = cm[i].copy()
    row[i] = -1
    confused_class = np.argmax(row)
    most_confused_for_each_class[class_names[i]] = class_names[confused_class]

print("\nMost likely confused class for each object type:")
for true_class, confused_class in most_confused_for_each_class.items():
    print(f"{true_class} -> {confused_class}")

cm_no_diag = cm.copy()
np.fill_diagonal(cm_no_diag, 0)

max_index = np.unravel_index(np.argmax(cm_no_diag), cm_no_diag.shape)
true_class_idx, pred_class_idx = max_index

print("\nMost confused overall (one direction):")
print(f"True class: {class_names[true_class_idx]}")
print(f"Predicted as: {class_names[pred_class_idx]}")
print(f"Count: {cm_no_diag[true_class_idx, pred_class_idx]}")

max_pair = None
max_pair_value = -1

for i in range(num_classes):
    for j in range(i + 1, num_classes):
        pair_value = cm[i, j] + cm[j, i]
        if pair_value > max_pair_value:
            max_pair_value = pair_value
            max_pair = (i, j)

i, j = max_pair
print("\nMost mutually confused pair:")
print(f"{class_names[i]} and {class_names[j]}")
print(f"Total confusion: {max_pair_value}")
```

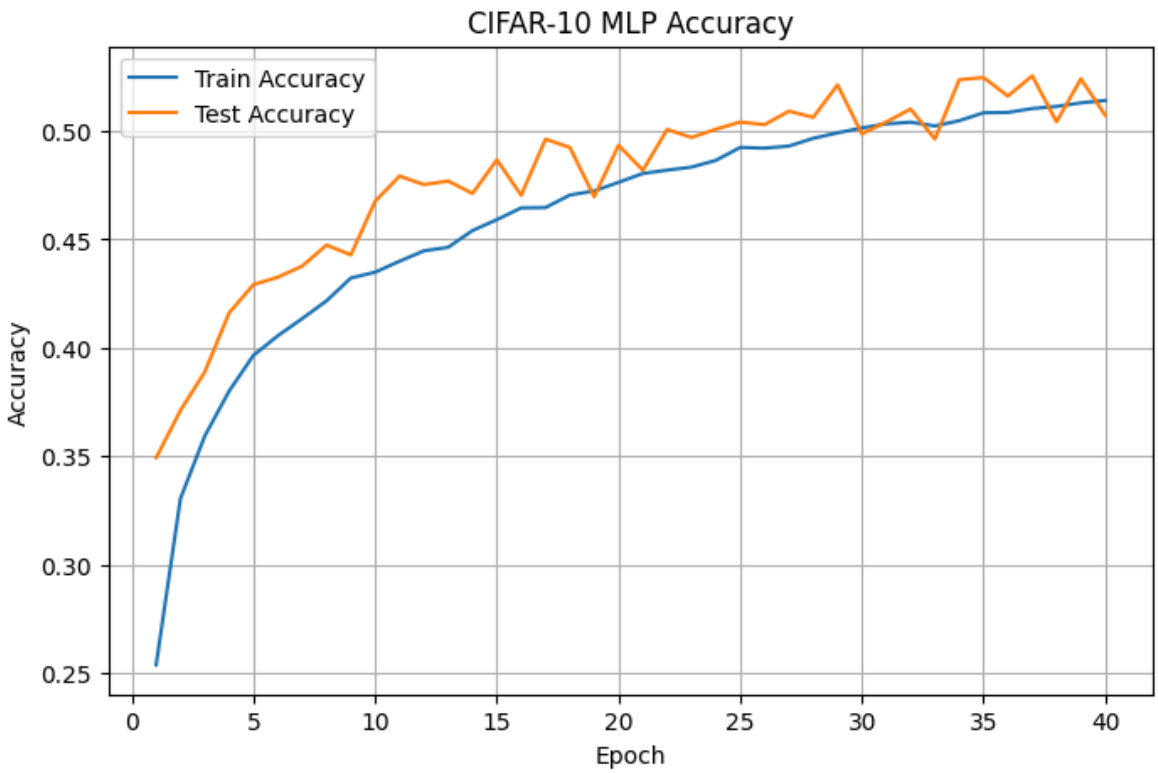
```
57.7%IOPub message rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub_msg_rate_limit`.

Current values:
ServerApp.iopub_msg_rate_limit=1000.0 (msgs/sec)
ServerApp.rate_limit_window=3.0 (secs)

100.0%
C:\Users\admin\anaconda3\envs\pytorch_hw\Lib\site-packages\torchvision\datasets\c
ifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python
or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a suba
rray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```

```
Epoch 1/40
Train Loss: 2.0281, Train Accuracy: 0.2539
Test  Loss: 1.8326, Test  Accuracy: 0.3494
----------------------------------------
Epoch 2/40
Train Loss: 1.8544, Train Accuracy: 0.3306
Test  Loss: 1.7485, Test  Accuracy: 0.3712
----------------------------------------
Epoch 3/40
Train Loss: 1.7824, Train Accuracy: 0.3595
Test  Loss: 1.6907, Test  Accuracy: 0.3890
----------------------------------------
Epoch 4/40
Train Loss: 1.7320, Train Accuracy: 0.3801
Test  Loss: 1.6593, Test  Accuracy: 0.4162
----------------------------------------
Epoch 5/40
Train Loss: 1.6950, Train Accuracy: 0.3966
Test  Loss: 1.6168, Test  Accuracy: 0.4292
----------------------------------------
Epoch 6/40
Train Loss: 1.6687, Train Accuracy: 0.4056
Test  Loss: 1.6024, Test  Accuracy: 0.4326
----------------------------------------
Epoch 7/40
Train Loss: 1.6423, Train Accuracy: 0.4135
Test  Loss: 1.5630, Test  Accuracy: 0.4377
----------------------------------------
Epoch 8/40
Train Loss: 1.6206, Train Accuracy: 0.4217
Test  Loss: 1.5563, Test  Accuracy: 0.4474
----------------------------------------
Epoch 9/40
Train Loss: 1.6007, Train Accuracy: 0.4322
Test  Loss: 1.5473, Test  Accuracy: 0.4429
----------------------------------------
Epoch 10/40
Train Loss: 1.5832, Train Accuracy: 0.4349
Test  Loss: 1.5009, Test  Accuracy: 0.4676
----------------------------------------
Epoch 11/40
Train Loss: 1.5647, Train Accuracy: 0.4400
Test  Loss: 1.4641, Test  Accuracy: 0.4792
----------------------------------------
Epoch 12/40
Train Loss: 1.5546, Train Accuracy: 0.4447
Test  Loss: 1.4780, Test  Accuracy: 0.4753
----------------------------------------
Epoch 13/40
Train Loss: 1.5423, Train Accuracy: 0.4464
Test  Loss: 1.4681, Test  Accuracy: 0.4769
----------------------------------------
Epoch 14/40
Train Loss: 1.5267, Train Accuracy: 0.4541
Test  Loss: 1.4702, Test  Accuracy: 0.4712
----------------------------------------
Epoch 15/40
Train Loss: 1.5141, Train Accuracy: 0.4591
Test  Loss: 1.4374, Test  Accuracy: 0.4867
----------------------------------------
```

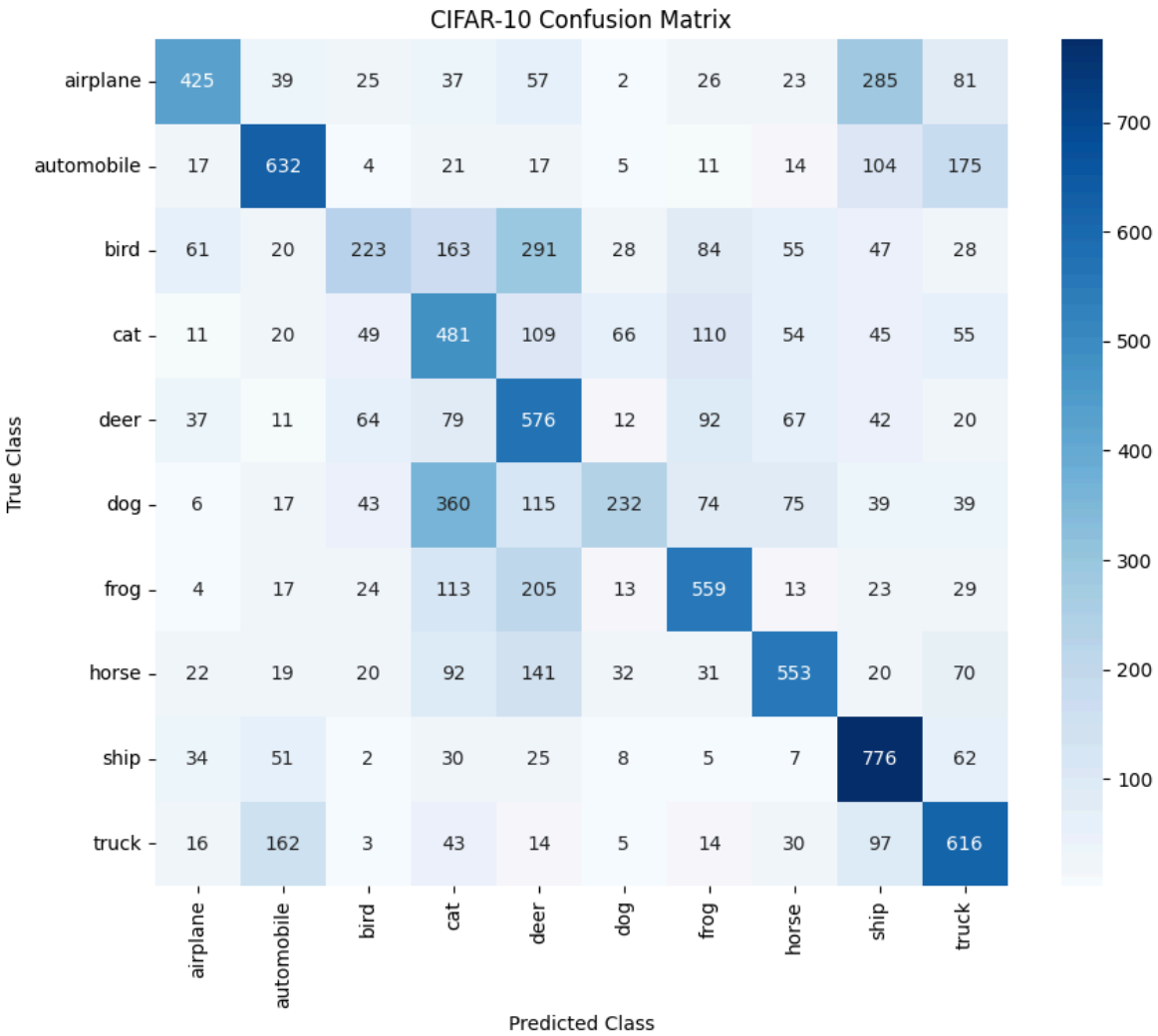
```
Epoch 16/40
Train Loss: 1.5035, Train Accuracy: 0.4645
Test  Loss: 1.4705, Test  Accuracy: 0.4704
----------------------------------------
Epoch 17/40
Train Loss: 1.4951, Train Accuracy: 0.4647
Test  Loss: 1.4092, Test  Accuracy: 0.4962
----------------------------------------
Epoch 18/40
Train Loss: 1.4774, Train Accuracy: 0.4705
Test  Loss: 1.4403, Test  Accuracy: 0.4924
----------------------------------------
Epoch 19/40
Train Loss: 1.4762, Train Accuracy: 0.4723
Test  Loss: 1.4892, Test  Accuracy: 0.4697
----------------------------------------
Epoch 20/40
Train Loss: 1.4653, Train Accuracy: 0.4763
Test  Loss: 1.4085, Test  Accuracy: 0.4934
----------------------------------------
Epoch 21/40
Train Loss: 1.4547, Train Accuracy: 0.4804
Test  Loss: 1.4414, Test  Accuracy: 0.4819
----------------------------------------
Epoch 22/40
Train Loss: 1.4454, Train Accuracy: 0.4819
Test  Loss: 1.3969, Test  Accuracy: 0.5007
----------------------------------------
Epoch 23/40
Train Loss: 1.4413, Train Accuracy: 0.4833
Test  Loss: 1.4235, Test  Accuracy: 0.4970
----------------------------------------
Epoch 24/40
Train Loss: 1.4365, Train Accuracy: 0.4864
Test  Loss: 1.4099, Test  Accuracy: 0.5008
----------------------------------------
Epoch 25/40
Train Loss: 1.4255, Train Accuracy: 0.4924
Test  Loss: 1.3837, Test  Accuracy: 0.5041
----------------------------------------
Epoch 26/40
Train Loss: 1.4209, Train Accuracy: 0.4921
Test  Loss: 1.4002, Test  Accuracy: 0.5029
----------------------------------------
Epoch 27/40
Train Loss: 1.4149, Train Accuracy: 0.4930
Test  Loss: 1.3705, Test  Accuracy: 0.5091
----------------------------------------
Epoch 28/40
Train Loss: 1.4073, Train Accuracy: 0.4966
Test  Loss: 1.3811, Test  Accuracy: 0.5063
----------------------------------------
Epoch 29/40
Train Loss: 1.4066, Train Accuracy: 0.4991
Test  Loss: 1.3464, Test  Accuracy: 0.5212
----------------------------------------
Epoch 30/40
Train Loss: 1.3940, Train Accuracy: 0.5014
Test  Loss: 1.3876, Test  Accuracy: 0.4987
----------------------------------------
```

```
Epoch 31/40
Train Loss: 1.3914, Train Accuracy: 0.5033
Test  Loss: 1.3832, Test  Accuracy: 0.5041
----------------------------------------
Epoch 32/40
Train Loss: 1.3888, Train Accuracy: 0.5040
Test  Loss: 1.3665, Test  Accuracy: 0.5101
----------------------------------------
Epoch 33/40
Train Loss: 1.3823, Train Accuracy: 0.5023
Test  Loss: 1.3948, Test  Accuracy: 0.4963
----------------------------------------
Epoch 34/40
Train Loss: 1.3757, Train Accuracy: 0.5047
Test  Loss: 1.3561, Test  Accuracy: 0.5236
----------------------------------------
Epoch 35/40
Train Loss: 1.3768, Train Accuracy: 0.5084
Test  Loss: 1.3366, Test  Accuracy: 0.5246
----------------------------------------
Epoch 36/40
Train Loss: 1.3664, Train Accuracy: 0.5085
Test  Loss: 1.3462, Test  Accuracy: 0.5161
----------------------------------------
Epoch 37/40
Train Loss: 1.3652, Train Accuracy: 0.5103
Test  Loss: 1.3489, Test  Accuracy: 0.5254
----------------------------------------
Epoch 38/40
Train Loss: 1.3597, Train Accuracy: 0.5113
Test  Loss: 1.3753, Test  Accuracy: 0.5042
----------------------------------------
Epoch 39/40
Train Loss: 1.3525, Train Accuracy: 0.5129
Test  Loss: 1.3421, Test  Accuracy: 0.5240
----------------------------------------
Epoch 40/40
Train Loss: 1.3548, Train Accuracy: 0.5140
Test  Loss: 1.3517, Test  Accuracy: 0.5073
----------------------------------------
```






```
Final Test Accuracy: 0.5073
```




```
Most likely confused class for each object type:
airplane -> ship
automobile -> truck
bird -> deer
cat -> frog
deer -> frog
dog -> cat
frog -> deer
horse -> deer
ship -> truck
truck -> automobile

Most confused overall (one direction):
True class: dog
Predicted as: cat
Count: 360

Most mutually confused pair:
cat and dog
Total confusion: 426
```

Model description

An MLP was trained on the CIFAR-10 dataset. The input images were flattened into 3072-dimensional vectors. The model used two hidden layers with 256 and 128 nodes, respectively, with ReLU activation after each hidden layer. A dropout layer with rate 0.3 was added after each hidden layer output, and L2

regularization with coefficient 0.0001 was implemented through weight decay in SGD. The final output layer had 10 units corresponding to the 10 CIFAR-10 classes.

Training result analysis

The training loss and test loss both decreased during training, while the training and test accuracy both increased steadily. The final test accuracy was about 50.7%, which is reasonable for a fully connected MLP on CIFAR-10. Since CIFAR-10 is a more complex image dataset and the model does not use convolutional layers, its performance is naturally much lower than on MNIST. The training and test curves remained relatively close, which suggests that dropout and L2 regularization helped reduce overfitting.

Confusion matrix analysis

The confusion matrix shows meaningful classification patterns. Several classes were classified relatively well, such as automobile, deer, frog, horse, ship, and truck. At the same time, there were several intuitive confusions between visually similar classes. For example, airplane was often confused with ship, automobile with truck, and cat with dog. These mistakes are expected because the model is a simple MLP and cannot capture local spatial features as effectively as a convolutional neural network.

Part (a)

The most likely confused class for each object type is:

airplane → ship automobile → truck bird → deer cat → deer deer → frog dog → cat frog → deer horse → deer ship → truck truck → automobile

Part (b)

The two classes most likely to be confused overall are cat and dog. This is reasonable because they share similar visual features, and a fully connected MLP has limited ability to distinguish fine local patterns in natural images.

In [ ]: